A Dual-Polarity Certifying Cascade for Equational Implication: Lean Proofs and Countermodels under Resource Bounds

Haobo Ma
ChronoAI Pte. Ltd.
Singapore

Wenlin Zhang
National University of Singapore
Singapore

Manuel Israel Cázares
Bytepro AI
Mazatlán, Mexico

Abstract

Proof-producing automation concentrates on establishing theorems, while counterexample generation is usually the province of separate model-finding tools. Equational implication demands both polarities at once: every input requires either a universal Lean 4 proof or an explicit Lean 4 countermodel, and an unverified failure on one side must never be treated as evidence for the other. We present a dual-polarity certifying architecture in which ordered superposition, structured algebraic search, bounded finite-model construction, and infinite-carrier witness generation all compile into first-class Lean 4 terms of opposite logical shape, each checked against the corresponding judge-generated `Goal` under a shared dependency policy. The unit of success is a typed Lean 4 certificate rather than a verdict: search state, heuristics, and unverified labels are compiled away before a result is submitted for acceptance, so defects in the untrusted search can reduce coverage but cannot by themselves carry an invalid certificate across the checking boundary. Ordered superposition is one producer inside this architecture.

We evaluate a frozen submission on 2,889 problems, obtaining 1,429 positive Lean 4 proofs and 1,460 negative Lean 4 witnesses, every one accepted by the judge. The unchanged submission subsequently produced accepted certificates for all 200 problems of an official set released after the freeze, a temporal holdout. Every reported certificate came from the local, non-stochastic cascade, with no language-model and no external-prover calls. The emitted certificates remain compact and inexpensive to check, which keeps production at this scale compatible with the deployment envelope. A harder research-tier corpus remains unsolved, delimiting the present coverage. The paper makes no completeness or cross-system superiority claim, and binds every empirical table to immutable ledgers.

1 Introduction

An equational theory of magmas has one binary operation and no assumed associativity, identity, or cancellation law. Given equations E_1 and E_2, the implication problem asks whether every magma satisfying E_1 also satisfies E_2. Proof-producing provers principally target the positive direction: they establish a theorem or reconstruct a proof found by an ATP. Counterexample generation is usually exposed through a separate model finder. Neither interface alone meets the implication problem's two-sided contract. Each input requires one of two mathematically opposed objects, and failure to find or validate one object is never evidence for the other polarity.

The SAIR Mathematics Distillation Challenge, organized by Damek Davis and Terence Tao with the SAIR Foundation, makes that gap operational [4, 18]. A true answer must contain a Lean 4 proof of the universal implication; a false answer must exhibit a magma in which the hypothesis holds and the conclusion fails. The same judge checks both forms. A plausible label, proof sketch, failed proof search, or failed model search receives no credit [17, 18].

We address this gap with a Lean-facing dual-polarity certificate architecture. Ordered superposition compiles to a positive equality term; finite and structured models compile to reflected existential witnesses; and infinite Austin models compile to explicit mathematical theorems and a separating tuple. All three paths target the judge-generated `Goal` and the same dependency policy. The unit of success is thus a typed Lean 4 artifact, not a verdict. Search state is compiled away, and judge acceptance is the only commit point.

The producers run as a resource-bounded cascade in one Python file, under both the per-problem Solo protocol and the globally budgeted Marathon protocol. Across the principal public, temporal-holdout, and distribution-drill evaluation, the frozen artifact produced 2,889 accepted artifacts: 1,429 positive proofs and 1,460 negative witnesses. Its bytes were frozen before the official stress set was released; the same bytes later produced 200/200 accepted certificates on that set. All reported accepted results are produced deterministically, with zero language-model and zero external-prover calls. The separate research-tier experiment in Section 7.6 records the boundary of this coverage.

This paper makes four contributions.

- A Lean-facing dual-polarity architecture makes positive proofs and negative witnesses first-class terms

under one judge-generated goal and dependency policy, with acceptance rather than a search verdict as commit.
- A certificate-preserving compiler translates superposition traces into compact core-Lean proofs. It slices the reachable recipe DAG, reconstructs substitutions and one-hole contexts, lifts equalities with `congrArg`, composes symmetry and transitivity, and emits shared ancestors once in topological order.
- Three negative compilers emit reflected finite tables, algebraically structured operations, and explicit infinite-carrier witnesses. The infinite path carries the full mathematical argument because exhaustive checking over $\mathbb{N}$ is unavailable.
- Under strict resource bounds, the implementation produced thousands of judge-accepted artifacts, including a temporal holdout produced by the frozen bytes; the evidence separates this result from completeness or cross-system superiority claims.

This architecture occupies a different point from existing positive-proof automation. Duper supplies proof-producing superposition inside Lean, while Lean-Auto, Lean on Vampire, and Isabelle reconstruction connect mature provers to proof assistants [3, 7, 9, 16]. Those systems establish the positive-proof baseline. The architecture studied here is complementary: a narrower equational engine becomes one producer in a two-polarity system that also generates checked finite and infinite models under a single-file deployment contract. Section 9 gives the full comparison.

Section 2 defines the Lean certificate architecture before Section 3 presents the Python search cascade that supplies it. Sections 4 and 5 describe the countermodel and proof searches; Sections 6 and 7 cover deployment and evaluation. We then state the reproducibility boundary and position the system relative to equational-theory, superposition, and proof-assistant work.

2 Lean Certificate Architecture

The architectural interface is a pair of Lean types, not a pair of Boolean labels. For each parsed input the judge generates a verdict-specific `Goal`. Suppressing the bound variables inside the identities, the two shapes are

$$Goal^{+} := \forall (G : \mathsf{Type})\ [\mathsf{Magma}\ G],\ E_1(G) \to E_2(G)$$

and

$$Goal^{-} := \exists (G : \mathsf{Type})\ (\_ : \mathsf{Magma}\ G),\ E_1(G) \wedge \neg E_2(G).$$

The solver submits a definition named `submission : Goal`; it never reconstructs a look-alike proposition from its own parser. Consequently, Lean elaboration checks the actual binder order, operation instance, and polarity chosen by the judge.

2.1 Positive equality compiler

The positive producer does not export its internal refutation. Search treats the negated goal $u \neq v$ as a negative unit clause and closes it when the two sides unify. Compilation then walks backward from that closure and retains only recipes reachable from the two final equality chains. The resulting DAG excludes irrelevant saturation state; a topological traversal emits every shared ancestor exactly once.

Each recipe records its source and target equations, directions, overlap position, and substitution. The compiler reconstructs the instantiated terms and the one-hole target context. It turns the source equality into an equality at the overlap with `congrArg`, selects symmetry when a recorded direction requires it, and joins the lifted source equality to the target equality by transitivity. Demodulation uses the same construction with one-way matching. Thus a search over a negative clause becomes a positive equality chain in core Lean, expressed as locally typed `have` bindings followed by an `exact` term.

Compilation balances source size against transparent replay. Printing every normalization micro-step repeats contexts and substitutions, whereas replacing the whole derivation by a large tactic call reintroduces tactic behavior and library imports. The compiler therefore emits one named equality per selected inference, with normalization represented by explicit transitivity as needed. If the direct printer is rejected, robust re-emission wraps individual steps in a small set of core closing alternatives and resubmits the same derivation; it does not rerun superposition or change the theorem. If search ends without a direct closure, a separate lemma-pool path may emit a bounded selection of the smallest proved consequences, but the resulting tactic must still close the exact judge goal or the row remains unanswered.

For example, from $x \diamond y = x$ the goal $(a \diamond b) \diamond c = a$ is compiled by putting the instance $a \diamond b = a$ in the context $q \mapsto q \diamond c$ and composing with $a \diamond c = a$:

```
import JudgeProblem
def submission : Goal := by
intro G _ h a b c
have e0 : (a ◇ b) ◇ c = a ◇ c :=
congrArg (fun q => q ◇ c) (h a b)
exact e0.trans (h a c)
```

Term ordering, unification code, and the refutation data structure do not occur in this artifact.

2.2 Negative witness compilers

A finite or structured search result is compiled into the carrier, a total Magma operation, a proof that every assignment satisfies E_1, and a concrete assignment falsifying E_2. For carriers through ten, `finOpTable` embeds a JSON-like Cayley table and `decideFin!` reflects the finite checks. Its parser extracts individual decimal characters, so it cannot faithfully

Table 1. Positioning against the closest automation architectures. "Negative" means an explicit countermodel certificate, not failure to prove.

System	Search locus	Checked positive result	Negative certificate	Deployment distinction
Duper [7]	Lean tactic, dependent type theory	Lean proof	no	general proof-assistant integration
Lean-Auto [16]	external E/Vampire/Z3	reconstructed Lean proof	no	multi-ATP interface
Lean on Vampire [3]	external Vampire	reconstructed Lean proof	no	imports mature ATP proofs
SLAM [9]	Isabelle λ-superposition	Isabelle proof	no	higher-order / Sledgehammer backend
EQT2-S2	single Python file, unit equations	positive Lean equality proof	finite or infinite Lean witness	dual polarity, no external prover, hash-bound holdout

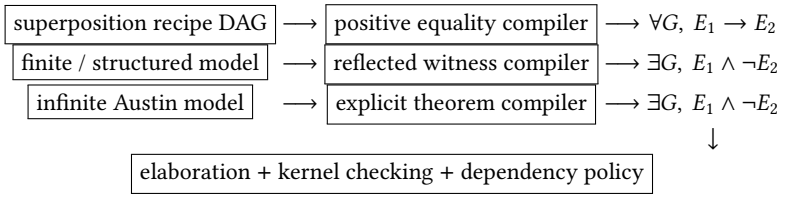


Figure 1. Three certificate-producing pipelines share one Lean acceptance boundary. Search-specific representations disappear before acceptance.

encode multi-digit entries. Larger finite carriers and algebraically structured families instead receive a transparent operation such as `submission.op`, defined by core arithmetic or a packed-table lookup. The submitted theorem refers to that operation directly, allowing Lean to evaluate the same structure without the table parser.

Infinite witnesses require a different compiler. For a recognized Austin pair it defines the selected operation on $\mathbb{N}$ (or the opposite operation), instantiates a fixed sequence of elementary lemmas proving E_1 universally, and supplies a small tuple proving $\neg E_2$. No exhaustive tactic can decide the universal premise over $\mathbb{N}$; the full mathematical argument is therefore part of the certificate rather than a Python-side claim.

2.3 Dependency policy and acceptance

The permitted imports are the judge's problem module and, for reflected finite witnesses, its finite-decision support modules; positive proofs and current infinite witnesses otherwise use core definitions and equality eliminators. The emitted source contains no admitted terms. The dependency inspector checks the transitive declaration and axiom closure of `submission` against the judge policy. Submission-local helpers are the documented way to package operations and lemmas: they are not global dependencies, but their transparent bodies are still elaborated, compiled, and traversed by the kernel. In particular, the allow-list inspects direct constants in the submitted term; the documented exemption for names in the `submission` namespace permits local auxiliary definitions but does not exempt their bodies from kernel checking. The policy controls dependencies, not the truth of definitions.

Acceptance is a commit operation. Python-side evaluation can discard a bad table or malformed term cheaply, but passing such a check records only a candidate. The solver commits a row only after the official judge has elaborated `submission : Goal`, kernel-checked it, and accepted its dependency closure. A rejected term has no evidential effect on either polarity; the next producer may run, or the row remains unanswered.

The positive compiler maintains a more precise replay invariant: for every retained equation $a = b$, the recipe DAG emits a Lean term whose type is exactly $a = b$ under instances of the input hypothesis. The base recipe is an input hypothesis instance. The overlap and demodulation cases follow from the recorded substitution, one-hole context, `congrArg`, symmetry, and transitivity. Dependencies precede users in the acyclic emission order, so induction over that order justifies the invariant; this metatheoretic argument is not itself mechanized. At the closure roots the two chains meet at the unified negative goal, yielding the requested positive equality. This argument specifies the compiler; the judge still checks each emitted instance and is the only trusted checker.

Table 2. Lean constraints, the certificate design each one forced, and the observed consequence. Every consequence restates a measurement bound elsewhere in this paper; the table introduces none of its own.

Challenge	Certificate response	Observed consequence
Saturation traces are too large	Reachable-recipe slicing, topological emission, and one binding per shared ancestor	Public and holdout certificate size distributions in Table 5
Broad tactic imports are too expensive	Core-only positive proof with explicit lemmas	Austin compilation fell from about 330 s to within 3.1 s
`finOpTable` cannot encode multi-digit entries	Transparent namespaced operation over the larger carrier	The official judge accepted the recorded larger-carrier witnesses
The searcher or printer may be wrong	Compile search state away; require a term of the exact judge type	Kernel-rejected frontier candidates remained unaccepted
The hosted toolchain changes	Keep certificates dependency-light and locally typed	Full frozen-artifact revalidation under Lean 4.33.1

3 Single-File Solver Architecture

3.1 Deployment contract

The submission contract requires one `solver.py` file no larger than the competition limit. Solver v2.5 is 189,504 bytes, and therefore remains below that limit while containing all search code and fixed data. In Solo mode the proxy sends one JSON problem and its budget to a fresh process and receives service requests over line-delimited JSON. In Marathon mode the runner supplies a JSONL manifest and an append-only output path. The same source supports both modes, selected by the runner-owned Marathon environment variable [17]. Section 2 defines the shared typed interface and explains why Python search remains outside the trusted boundary.

At both protocol entry points we normalize the input operation symbol. The official problem format permits either an asterisk or a diamond, but the solver's term parser has one internal token. Mapping the asterisk to $\diamond$ before parsing preserves the semantics and is operationally necessary; Section 6.2 records the failure that revealed it.

3.2 Cheapest-first cascade

The Solo cascade is ordered by expected cost and by which branch benefits from remaining time.

Stage A: shallow countermodels. The solver first tries tiny brute-force tables, scalar linear and affine operations, vector-linear families, low-degree polynomial operations, and recognized infinite-carrier Austin models [1]. The expanded structured families add about 0.8 seconds on a true input in the recorded development measurement. No irregular finite-model search is performed yet.

Stage B: direct true certificates. Syntactic singleton collapse and substitution-instance checks produce direct proofs. A specialized singleton-forced prover then receives an eight-second cap, although applicable cases normally close during its initial search. The general superposition engine follows with a quick budget $\min(6, \max(1, T/600))$ seconds for a process allowance T; this is six seconds under the recorded Solo allowance. Most provable public instances terminate within this cap.

Stage C: irregular models. A finite-model search over carriers four through ten receives eight seconds. It propagates the universal instances of the hypothesis and Latin-square constraints rather than enumerating complete tables.

Stage D: deep false search. Only residuals reach heavier polynomial and vector families, capped at sixty seconds, followed by another finite-model pass capped at one hundred twenty seconds. Deep false search precedes the deepest proof pass so that a difficult false row does not consume most of its allowance in proof saturation.

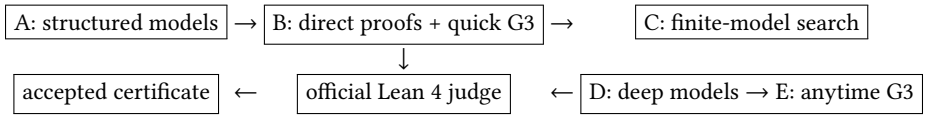
Stage E: proof search. The ordered-superposition engine follows the completion tradition of equational provers [2, 12] and receives $\min(600, \max(20, T/6))$ seconds, which is six hundred seconds under the Solo allowance. The engine repeatedly raises term-size and variable caps. If no direct refutation is returned, a small pool of derived equalities can be emitted as exact lemmas followed by a kernel-checked closing tactic. Solo also contains a last-resort language-model fallback, but it was never called for any archived accepted row; its measured history is reported in Section 7.

Every stage independently verifies its candidate before the solver stops. A rejected certificate proceeds to the next strategy; a successful search without a valid emitted term is not counted as a solution. This control flow distinguishes search coverage from certificate robustness.

3.3 Representation and dispatch

Both branches share a small non-associative term representation. Parsing produces a binary tree with variables at leaves; no reassociation or commutative normalization is performed. Alpha-renaming maps variables to a canonical local order for cache keys and family matching, but emitted proofs retain a map back to the judge's binder order. Operation count, variable support, depth, and tree size are computed once and reused by scheduling, countermodel probes, and proof search.

Dispatch relies on positive recognizers. A direct proof stage returns only after constructing a substitution or equality chain. An Austin or central-groupoid stage returns only when the hypothesis matches its stored schema and a concrete evaluation separates the goal. A family that is merely



Each search box is untrusted and may return no candidate. Every Solo arrow to the left carries a complete positive or negative certificate; only the judge's accepted response terminates the cascade. Marathon executes the same boxes in two fairness passes and defers judging until the append-only answer file is closed.

Figure 2. Cascade and trust boundary. The distinctive interface is dual-polarity: proof search and model search share the same fail-closed commit point.

```
normalize both input equations
for stage in [A, B, C, D, E]:
candidate := run(stage, remaining time)
if candidate = none: continue
reply := judge(candidate.polarity, candidate.code)
if reply = accepted: return
return unanswered
```

Figure 3. Solo scheduling pseudocode. Search never exports a cached Boolean verdict; a rejected candidate has no effect on the opposite polarity.

likely from equation identifiers is not selected: identifiers are useful for diagnostics, but the solver matches parsed laws. This property supports independent replay and handles inputs whose labels or variable names differ from those in the public files.

Candidate validation is stratified. Low-cost Python evaluation removes malformed tables, wrong witnesses, and printer bugs before a judge call. Judge feedback then closes the semantic and dependency obligations. The development harness records both outcomes: a "found" countermodel rejected by Lean 4 indicates an emitter defect or an encoding mismatch, whereas the absence of a Python witness within a deadline indicates a coverage limitation. This distinction informed the large-carrier encoding correction in Section 4.5.

3.4 Solo and Marathon

Solo gives each row a fresh process and a fixed allowance. Marathon instead requires triage across a whole manifest. The batch path stably sorts problems by a structural cost derived from operation occurrences, variable count, and equation length. It then runs two passes. The first gives every row the shallow stages before any residual receives deep work. The second revisits only residuals. A fair cap is recomputed from remaining wall-clock time divided by remaining rows and clamped to the documented per-row interval. At most fifty-five percent of a residual's current slice goes to the false branch; the proof branch receives the rest.

Marathon outputs constitute durable state. Each answer is one JSON line, flushed and synchronized before the next row. If the runner terminates the process, all complete earlier lines remain scoreable. A process-level timer bounds legacy search loops that predate the batch driver. The batch path leaves a residual unanswered when its certificate-producing stages are exhausted.

3.5 Cascade invariants and failure isolation

Stage ordering does not assert that either polarity is more likely. Instead, it minimizes expected certificate cost while reserving time for both branches. A structured false witness is attractive early because checking a closed operation is usually cheaper than saturation. Direct true patterns are equally attractive because they avoid both table construction and general proof search. Measurements showed that most provable rows close near the start of saturation, which motivates placing the quick G3 pass before irregular model search. Deep false search precedes deep G3 only after every low-cost recognizer has failed.

The implementation maintains a cascade invariant: before entering a stage, the solver has no accepted certificate, and every earlier candidate has either been absent, failed local construction, or been rejected by the judge. It does not maintain the stronger and generally invalid invariant that earlier stages have ruled out their polarity. Failure to find a finite model does not support truth; failure to derive the goal does not support falsity. Each stage can therefore be analyzed and tested as a partial certificate producer, and the composition is sound whenever the judge is sound, regardless of the producers' coverage.

Exceptions are contained at strategy boundaries in the Marathon path so that a malformed residual cannot discard previously completed rows. Solo has a fresh process per row and can rely on the runner to record a crash. Within a strategy, deadlines are checked at natural allocation points: carrier changes, model-search decisions, deepening rounds, and given-clause iterations. The Marathon alarm provides a final outer bound for code paths whose internal checks are too sparse. Timeout returns "no candidate" rather than a verdict.

Caches follow the same lifetime discipline. Parser and static-law data live for the process. A superposition run owns its unifier, normalization, term, and clause caches, which are cleared before another problem. Marathon could in principle share more proof state across rows, but the current implementation does not rely on cross-problem lemmas. This

keeps Solo and Marathon certificate semantics aligned and limits memory growth under a long manifest.

4 Countermodel Construction

For a false implication the solver must find an operation satisfying all assignments of the hypothesis and at least one assignment falsifying the goal. It evaluates candidate families in Python and retains a concrete witness; the three negative certificate shapes and their common acceptance boundary are defined in Section 2.

4.1 Coefficient matching over structured families

Represent a magma term by the formal coefficients induced by a candidate operation. For

$$x \diamond y = ax + by \pmod{n},$$

each leaf contributes a coefficient obtained by multiplying edge labels along its path. Equality of two terms for all assignments is therefore equality of their coefficient vectors modulo n. This gives an exact, inexpensive test for whether the hypothesis is an identity of the candidate magma. The same representation quickly locates an assignment on which the goal coefficients differ. The scan includes composite moduli and its shallow range extends to the largest carrier admitted by the certificate-instance guard.

Affine operations add a constant term. Matrix-linear operations replace scalar coefficients by matrices over small finite fields. These searches include two-dimensional vectors over the three-element field and three-dimensional vectors over the two-element field. Polynomial candidates are evaluated on full assignments when the carrier and hypothesis arity make that feasible. The deep pass adds a quadratic grid, a two-dimensional family over the five-element field, and sampled polynomials of degree at most three. An instance-count guard prevents a mathematically compact operation from producing a certificate whose exhaustive hypothesis check exceeds the judge budget.

These families provide both search efficiency and structured certificates. They find many witnesses faster than a generic table search while preserving semantic structure in the emitted certificate. For example, a linear countermodel can be represented by its arithmetic operation rather than by a large opaque table. The judge still evaluates the hypothesis and the failing goal instance, so coefficient matching remains an untrusted search optimization rather than a proof rule.

Coefficient evaluation is implemented recursively. A variable leaf maps to its basis vector. At an internal node, the coefficient vector is $a\vec{u} + b\vec{v}$ for child vectors $\vec{u}$ and $\vec{v}$; affine families carry one additional constant coordinate. Thus the universal hypothesis test does not enumerate assignments. If the two sides have different vectors, the candidate is immediately rejected as a model of the hypothesis. If the hypothesis vectors agree and the goal vectors differ, a separating assignment can be sought in the small module. Full evaluation is still applied before emission, which protects the certificate path from an error in this symbolic shortcut.

Vector-linear families use exactly the same recursion with matrices acting on coordinates. They are useful because scalar coefficient coincidence may hold in every small modulus even when non-commuting linear actions distinguish the goal. Polynomial operations abandon the linear invariant and instead use compiled term evaluators over bounded carriers. The family ordering therefore progresses from symbolic coefficient equality to increasingly expensive full evaluation, while presenting a uniform result to the certificate printer: a carrier, a total binary operation, and a goal witness.

4.2 Irregular finite models

Some countermodels have no low-degree algebraic description. The finite-model stage treats the Cayley table as a constraint problem. For a proposed carrier size, table cells begin unassigned. Ground instances of the hypothesis watch the cells on which their two terms depend. Assigning a cell may simplify a term, force another cell, or violate an instance. The search uses a minimum-remaining-values choice among table positions and abandons partial tables as soon as a hypothesis instance becomes unequal.

Many identities imply that rows, columns, or both must be permutations. A law in which a variable occurs once on one side and appears under the operation in a particular argument position can expose such a constraint. The solver infers the applicable Latin flags, rejects duplicate row or column values, and forces naked singletons. For one residual requiring an eight-element quasigroup, this propagation changed the outcome from no witness within a one-hundred-twenty-second attempt to a witness in 0.3 seconds. The production shallow search remains restricted to carriers four through ten and an eight-second cap; the deep pass revisits the same carrier interval with a longer cap.

Before emission, the completed table is checked by a separate evaluator over all hypothesis assignments and a stored goal-falsifying assignment. The Lean 4 certificate then reconstructs the table operation and invokes the judge's finite-decision tactic. The duplicated checks reduce wasted judge calls, while only the latter determines acceptance.

The partial-table solver maintains two invariants. Every assigned cell agrees with all hypothesis instances that have become ground, and every inferred Latin constraint agrees with the assigned portion of its row or column. A watch list connects a cell to the term instances whose evaluation may advance when that cell is filled. Propagation reaches a fixed point before the next decision. At a complete table, the first invariant implies that exhaustive reevaluation of the hypothesis should succeed; the separate final evaluator checks this implication and also searches the goal assignments. Search backtracking may be unsound or incomplete as an algorithm without threatening an accepted answer, because the table is

subsequently evaluated from scratch and then checked once more in Lean 4.

Carrier growth is likewise heuristic. The shallow and deep passes enumerate the documented carrier interval, but the solver does not infer that absence of a model there implies truth. It passes the row to the next proof or model family. This behavior implements the paper's no-completeness boundary.

4.3 Central groupoids and the E168 residual family

The evaluation-distribution drill identified a family of twelve residuals whose hypothesis is the central-groupoid law. Central groupoids have a well-known combinatorial structure studied by Knuth [11]. The natural constructions tried by the structured stages satisfy both the hypothesis and the relevant E168 conclusions, so they do not separate these equations. Increasing the generic finder budget did not reliably recover the missing witnesses.

The solver therefore includes one explicit non-natural central groupoid of order nine. The table was checked against each relevant hypothesis and goal, then stored once as a fixed witness family. When the hypothesis matches up to variable renaming and a goal is separated by the table, the solver emits the ordinary finite-table certificate. This stage is not a classification of central groupoids and makes no claim about minimal order. It provides a specific, auditable witness for a recurring algebraic family: the official runner accepted the emitted certificates for the twelve drill residuals.

4.4 Infinite carriers for Austin pairs

Finite satisfiability can conceal false general implications. Several pairs catalogued in the Equational Theories Project are true over the finite magmas searched by standard methods but false in general. For recognized Austin-pair hypotheses, the solver selects a fixed operation on $\mathbb{N}$, or its opposite, and a small tuple falsifying the goal. The certificate proves the hypothesis through a fixed sequence of elementary lemmas and closes the negated goal with the concrete tuple.

Unlike finite decision, this search cannot discharge the universal premise by enumeration. Section 2 gives the explicit theorem compiler; Section 6.1 records the import and compilation costs that motivated its core-only form.

4.5 Certificate shapes and the allow-list

Section 2 specifies the reflected finite shape, the single-digit limitation of `finOpTable`, the transparent namespaced operation used for larger carriers, and how the dependency inspector treats local definitions. The official judge accepted the recorded larger-carrier certificates in this alternative shape.

5 Proof-Producing Ordered Superposition

The true branch operates in unit equational logic. It assumes one universally quantified identity and attempts to derive the goal identity. The engine is inspired by ordered superposition and unfailing completion as implemented in first-order provers, but is specialized to one binary symbol, unit clauses, and the need to reconstruct a small positive Lean 4 proof.

5.1 Terms, ordering, and inference

Terms are immutable trees over variables and the binary magma symbol. A Knuth–Bendix ordering with unit symbol weights determines maximal equation sides and orients usable rewrite rules. Superposition overlaps an oriented side with a non-variable subterm of another maximal side, computes a most general unifier, and constructs the resulting equality. Ordering checks are repeated after unification, because substitution can change comparisons. Equations that cannot be oriented remain available for symmetric inferences.

More precisely, all variables in the two premises are renamed apart. For equations $l = r$ and $s = t$, a non-variable position p of s, and $\sigma = \mathrm{mgu}(l, s|_p)$ (with an occurs check), the implemented positive inference has the schematic form

$$\frac{l = r \qquad s = t}{s[r]_p\sigma = t\sigma}. \tag{SP}$$

The selected source direction must be KBO-decreasing after instantiation unless the premise is unorientable; for an unorientable premise the direction is eligible only when its instantiated right side is not greater. The selected target side must be maximal after instantiation, i.e., $t\sigma \not> s\sigma$. The implementation considers both sides exactly when static KBO cannot orient the premise. With unit weights, KBO first checks the variable-occurrence condition, then total tree size, and finally lexicographic order for the single binary symbol; fresh goal constants are ordered by their names.

Demodulation is the matching instance of (SP). If $l\theta = s|_p$, it rewrites s to $s[r\theta]_p$ only for an oriented rule or when the concrete rewrite is KBO-decreasing. Forward demodulation normalizes new equations; a newly active oriented rule backward-demodulates and retires reducible active equations. Equations reduced to $q = q$ are deleted. For a negative goal $u \neq v$, the corresponding rule is

$$\frac{l = r \qquad u \neq v}{u[r]_p\sigma \neq v\sigma}, \tag{NEG}$$

with the same source condition and a maximality check on the selected negative side. A negative clause closes when its two sides unify. These rules describe the implementation rather than asserting a new completeness theorem.

The goal is represented internally as a negative unit clause $u \neq v$ over fresh constants. Active equalities can superpose into either side; ordinary demodulation simplifies the clause. If its sides unify, the negative clause is refuted. This internal negative representation is convenient for search, but the emitted proof does not ask Lean 4 to validate a refutation

calculus. It instead reconstructs a positive equality chain ending at the original goal.

Forward demodulation normalizes each generated equality by current rules. Backward demodulation revisits active equations when a new rule can simplify them. Symmetric alpha-normalized keys remove variants. A given-clause loop interleaves a weight heap with an age queue; later deepening rounds alter the variable penalty and age ratio so that a proof blocked by one selection policy is not permanently starved.

5.2 The tautology-deletion failure

An early version of the prover did not resolve one public residual, `hard3_0314`. The official-runner attempt found no proof after 810 seconds even though an external derivation suggested that the necessary clauses were within the configured size bound. Instrumentation indicated that throughput was not the primary cause. When demodulation transformed an equation into $s = s$, the step constructor returned no proof node, and the caller retained the partially rewritten input equation. Backward simplification had the same behavior.

Consequently, many small instances of the hypothesis remained live. Their weights made them attractive to given-clause selection, so they repeatedly displaced clauses on the useful derivation. The correct simplification rule is deletion: a demodulated tautology contributes nothing to saturation. The fix retires the active clause before testing the rewritten result and drops a new clause whose sides become equal.

With that change, the prover found the residual proof in 0.6 seconds in its direct measurement, and the official-runner path produced an accepted certificate in 14.6 seconds of wall-clock time. These measurements illustrate the coupling between simplification and selection rather than establishing a general speed ratio. Retaining a tautology as a live clause changed the effective search strategy enough to prevent the final public proof from being found within the earlier attempt.

5.3 Indexing and allocation control

Demodulation candidates are retrieved through a discrimination tree keyed by a term's linear skeleton. The index returns rules whose left sides may match a subterm; matching and the ordering check filter false positives. Normal forms are cached and versioned by the rewrite-rule set. Term sizes, variable multisets, preorder traversals, and eligible positions are cached on immutable equations.

The unifier returns a triangular substitution rather than eagerly applying it to every binding. A memoised substitution routine resolves chains while sharing unchanged subtrees. Before constructing a superposition result, the engine estimates instantiated size from static term sizes and per-variable counts. Overlaps that cannot fit the current cap are rejected before allocating their terms. Unifier results are cached by the renamed overlap pair. These changes preserve the inference trace at fixed bounds while reducing the work per surviving clause.

The passive set stores recipes—source equation, target equation, direction, and overlap position—rather than full instantiated proof objects. The selected recipe is materialized when it becomes given. Under the hosted memory limit, this representation permits a large passive frontier without duplicating every term and ancestry chain. Caches and negative-goal storage are bounded and cleared between prover invocations.

5.4 Anytime deepening

A fixed size cap is sensitive to the selected bound: saturation at a small cap can terminate with time remaining even when one slightly larger intermediate would finish the proof. The deep pass therefore runs a schedule of increasing term-size and variable caps. If a round saturates, its unused time moves forward. If the final configured round saturates, caps continue to grow within the overall deadline. The last strategies reduce the variable penalty and increase age pressure, approximating a different prover configuration without maintaining another implementation.

The procedure remains budget-bounded and machine-load sensitive. It is not a new completeness result for ordered superposition, nor a proof that the chosen ordering is fair under every cap. The evaluation in Section 7 measures this implementation on fixed corpora.

5.5 Replay into the Lean 4 kernel

Section 2 gives the complete compiler: reachable-proof slicing, substitution and context reconstruction, topological emission, positive-chain conversion, robust re-emission, the replay invariant, and a worked Lean term. The search engine described in this section supplies its recipe DAG; it never asks Lean to trust the ordering or refutation calculus.

6 Engineering for the Hosted Environment

6.1 Toolchain migration

Section 2 gives the final core-and-judge-module dependency policy. Historically, the public harness used Lean 4 4.30.0-rc2, while the announced hosted verifier used Lean 4 4.32.0 with the corresponding Mathlib release. Most emitted certificates already followed that policy. The exception was the infinite-carrier Austin family, whose tactic import made compilation sensitive to the larger library environment. In a compatibility run, one such certificate required about 330 seconds of compilation.

Rather than increasing the solver-side timeout, we rewrote the Austin certificate in positive form, replaced the broad tactic dependency with explicit core lemmas, and re-emitted the witness. Its Lean 4 4.32 compilation then completed

within 3.1 seconds. A stratified corpus of 120 accepted certificates, covering every emission family and all fourteen former v1 residuals, compiled 119 of 120 under the exact announced toolchain, the single non-pass being the pre-rewrite witness itself at an external 120-second phase limit that the judge's own 300-second configuration clears. That corpus characterizes the migration-era certificate set rather than the frozen submission's current output, which for the same problem imports only the judge module and is accepted in under five seconds. The judge support modules compiled unchanged; the run is archived as an operator attestation (`results/lean432_corpus/`). When the organizers subsequently upgraded the judge to Lean 4 4.33.1 after the kernel-soundness review, the frozen artifact was revalidated in full under the upgraded judge — official harness, all six public sets, the distribution drill, and the Marathon manifest — with the resulting ledgers committed and hash-bound (`results/lean4331_revalidation/`). Neither measurement substitutes for hosted scoring.

6.2 Input encoding

The official judge normalizes both operation spellings when it builds the Lean 4 problem. The runner, however, passes the original problem text to the contestant process. An earlier solver normalized only along one internal path. When the Stage 1 evaluation splits were supplied in their published asterisk encoding, the solver crashed before certificate search and recorded zero accepted rows out of 800.

The frozen solver normalizes both equations at both the Solo and Marathon intakes. The same drill then recorded 800 accepted certificates with full agreement against the published answers. Thus, verified output does not address failures in an unverified parser; normalization must occur at the protocol boundary, before any dispatch or caching.

6.3 Resource and filesystem constraints

The hosted-style sandbox has a read-only submission mount, a bounded process count, bounded memory, no direct network, and a small writable temporary filesystem. The solver consequently uses the Python standard library, keeps all static data in the single source file, does not spawn external provers, and does not assume repository access. Bounded passive queues and caches keep the superposition engine below the sandbox memory ceiling observed in long false saturations. The process-level timer also prevents a deeply nested legacy loop from overrunning its Marathon slice.

Marathon's output file is the only durable mutable state supplied by the runner, so per-row flush and synchronization are required for correctness; a managed workspace whose default artifact directory was unwritable was resolved by redirecting judge artifacts to temporary storage, leaving judge sources untouched.

Table 3. Canonical artifact identity. The freeze timestamp precedes the official stress-set release dated 2026-08-28.

Field	Canonical value
Solver	v2.5, 189,504 bytes
SHA-256	`f2392533c9f4c03b292be80bc6d12e98 e5254cc4861d1cc4b227957ad5ed89b4`
Freeze	`7b636bc3c3e8f4eed80fea1809ba940af3c466f0`, 2026-08-26 21:31:32 UTC
Local judge	`2848228ff490422442878fd6f5abaf4cfa95257d`; Lean 4.30.0-rc2, Mathlib `896cc56a`
Hosted recheck	`13648682a5553717ea91b86513ed140b39160cf5`; Lean 4.33.1, Mathlib `0df444a3`

7 Evaluation

7.1 Method and provenance discipline

Unless marked hosted, measurements in this section are local runs of the official runner and Lean 4 judge. The main regression used official repository revision `2848228ff490...`; the harness completed without errors on the operator host. Table 3 is the sole artifact identity used by every result below. Each result file is bound by the repository provenance manifest; short hashes in later tables are prefixes of the full SHA-256 values in `PROVENANCE.json`.

The reported wall-clock time is the sum of per-row runner measurements, not elapsed parallel makespan and not a hardware-independent cost model. Solver stages with deadlines may take different paths under heavier load. "LLM" counts language-model calls attributed by the runner. All reported accepted results are produced deterministically, with zero language-model and zero external-prover calls.

The Solo path retains a fallback that calls a language model only after the certificate stages; Marathon disables it, and any Solo proposal would still pass through the same printer and judge. No archived accepted row invoked it.[1]

7.2 Six public sets

Table 4 reports the final six-set regression. All rows were accepted, every accepted row had one judge call, and no row used the fallback model. The sum of the set sizes is 1,889. The earlier frozen v1 solver accepted 1,875 of those rows; v2 adds the structured false stages and G3 proof engine described above. We report v1 only as development context, not as a comparison against another entrant.

The totals establish regression coverage of these public artifacts only. The private evaluation set is separate, and public-set tuning is possible. We do not treat Table 4 as a hidden-set estimate.

[1] A separate historical measurement of that configuration, before its optimization, is recorded in `PROVENANCE.json`: the organizer-pinned model settled none of six residuals and varied between first attempts under fixed temperature and seed. It diagnoses one configuration and supports no general claim.

Table 4. Local official-runner regression for solver v2.5. "Wall" is the sum of per-row wall-clock seconds. Each row names its immutable ledger and SHA-256 prefix.

Set	Accepted	LLM	Judge	Wall (s)	Ledger provenance
`sample_20`	20/20	0	20	67.53	results/v2_sample_20_official_2848228.json `b17eaff452d2...`
`sample_200`	200/200	0	200	964.12	results/v2_sample_200_official_2848228.json `dcf830de6199...`
`hard1`	69/69	0	69	559.81	results/v2_hard1_official_2848228.json `7724566d1ceb...`
`hard2`	200/200	0	200	1250.67	results/v2_hard2_official_2848228.json `3e68548945d4...`
`hard3`	400/400	0	400	2857.38	results/v2_hard3_official_2848228.json `c9f98750e018...`
`normal`	1000/1000	0	1000	4128.3	results/v2_normal_official_2848228.json `27c756482d15...`

7.3 Certificate and checking cost

Table 5 first counts the accepted Lean artifacts in the principal evaluation and then aggregates UTF-8 source bytes and accepted judge-event time from the bound public and hold-out ledgers. "Check" includes judge orchestration around Lean compilation and kernel checking; it is not prover search time. Size and time distributions are separated by polarity because reflected countermodels and equality-chain proofs have different shapes.

Peak memory was not recorded by the official runner and is therefore not reported. The bounded recipe queues described in Section 5 are a design response to the hosted limit, not a substitute for measured resident set size.

7.4 Component-level interpretation

The superseded v1 baseline left fourteen public residuals: eleven true implications without a proof inside its budgets and three false implications without a model in its available families and carriers. The v2 development was organized around those failure classes. G3 plus the tautology-deletion correction supplies certificates for the true residuals; the large-carrier, Latin-propagating, structured, and infinite-carrier paths supply witnesses for the false residuals. Replaying the residual set alone records fourteen accepted certificates, whereas the six complete sets provide a regression check that stage ordering and printers preserve the previously accepted cases.

This history is diagnostic, not a controlled ablation study. Several changes were introduced together, and a row may now be solvable by more than one stage. The paper therefore attributes mechanisms only where the engineering notes contain a direct replay, such as `hard3_0314` for tautology deletion and `hard1_0062` for Latin propagation. It does not assign an aggregate percentage gain to individual components.

The order of the cascade also affects the observed wall totals. A true row may incur the shallow false-family cost before receiving a quick proof; a false row may incur direct proof checks before finite-model search. These costs are intentional because the early stages reduce resource use across both branches: inexpensive structured countermodels avoid proof saturation, while direct proof patterns avoid table search. The six-set totals measure the integrated policy, not isolated engine speed.

7.5 Distribution drill, Marathon, and hosted playground

The official-distribution drill uses the four published Stage 1 evaluation splits that were named as Stage 2 scoring categories. Each split contains published ground truth and uses the asterisk input encoding. Table 6 reports full local judge acceptance and full agreement with those labels. Stage 2 does not reuse these problems, so the drill measures format and distribution readiness rather than evaluation transfer.

The canonical Marathon measurement used the official batch runner and its `normal_100` manifest. All rows were accepted without tokens. The hosted-playground row is different in kind: it was run through the official playground interface on the hosted judge and is recorded as an external operator attestation, because the playground emits no repository-committable ledger. Across the four evaluation categories the attested runs record acceptance of every attempted problem with no rejected or errored rows and no language-model calls. It is explicitly not an official leaderboard score.

The hosted record further reports means by verdict: 2.69 seconds for false rows and 7.08 seconds for true rows, with maxima 6.5 and 23.8 seconds, respectively. Those values include hosted orchestration and certificate checking as exposed by the playground, and should not be compared directly to the local prover-only benchmark below.

Table 5. Accepted Lean artifacts and certificate costs. Panel (a) binds each empirical corpus to its source; the total is their arithmetic sum. In panel (b), entries are median / 95th percentile / maximum. Public and holdout statistics derive from `results/paper_certificate_metrics.json`, SHA-256 `56f233426fd63009...`; its embedded source hashes include the holdout ledger `c741609293d136c6...`.

(a) Accepted Lean artifact production

Corpus	Positive Lean proofs	Negative Lean witnesses	Total	LLM calls	External prover	Ledger provenance
Six public sets	929	960	1,889	0	0	results/paper_certificate_metrics.json `56f233426fd63009...`
Temporal holdout (official stress set)	100	100	200	0	0	results/stage2_stress_test_results.json `c741609293d136c6...`
Published distribution drills	400	400	800	0	0	results/official_distribution_drill/*.json `6647e3a13fc0...` / `2025ca90a892...` / `b931bbf33007...` / `d7ef36ee24f7...`
Total	**1,429**	**1,460**	**2,889**	**0**	**0**	Derived from the three hash-bound rows above

(b) Certificate source and checking cost

Corpus	Verdict (n)	Certificate bytes	Check time (s)	End-to-end wall (s)
Public	false (960)	306 / 498 / 6,885	2.20 / 8.85 / 41.74	2.30 / 13.43 / 41.95
Public	true (929)	494 / 1,392 / 10,766	1.43 / 6.98 / 23.21	4.26 / 15.43 / 35.29
Temporal holdout	false (100)	323 / 551 / 551	2.64 / 14.89 / 17.92	2.77 / 15.69 / 18.04
Temporal holdout	true (100)	500 / 1,406 / 12,485	1.72 / 6.13 / 10.04	5.91 / 12.12 / 66.49

Table 6. Additional measurements. Local drill and Marathon rows name their ledger files and SHA-256 prefixes. The hosted row is an operator attestation recorded in the `hosted_playground_measurement` entry of `PROVENANCE.json`; the playground interface did not emit a repository ledger file.

Measurement	Accepted	LLM/tokens	Wall statistic	Ledger provenance
`evaluation_order5`	200/200	0 calls	sum 2047.27 s	results/official_distribution_drill/evaluation_order5_results.json `6647e3a13fc0...`
`evaluation_extra_hard`	200/200	0 calls	sum 17307.45 s	results/official_distribution_drill/evaluation_extra_hard_results.json `2025ca90a892...`
`evaluation_hard`	200/200	0 calls	sum 2228.57 s	results/official_distribution_drill/evaluation_hard_results.json `b931bbf33007...`
`evaluation_normal`	200/200	0 calls	sum 1019.91 s	results/official_distribution_drill/evaluation_normal_results.json `d7ef36ee24f7...`
Marathon `normal_100`	100/100	0 tokens	full default budget	results/marathon/normal_100_summary.json `d422d4ad6703...`
Hosted playground (`evaluation_normal`)	200/200	0 calls	mean 4.89 s/problem	PROVENANCE.json:hosted_playground_measurement `artifact 7916cbc2...`

7.6 Temporal holdout and research frontier

The solver artifact in Table 3 was frozen at 21:31:32 UTC on 2026-08-26. The organizers released the official Stage 2 stress set on 2026-08-28, after that freeze. The set was announced as mirroring the final leaderboard composition: four categories of fifty rows, each containing twenty-five true and twenty-five false implications. The byte-identical solver then produced 200/200 accepted certificates with zero language-model calls in 22 minutes 25 seconds. Its per-category verdict counts match the announced composition, although the set does not publish per-item reference labels. Its ledger has SHA-256 prefix `c741609293d136c6`; the full digest is bound in `PROVENANCE.json`.

The temporal ordering, immutable solver hash, and absence of post-release parameter changes make this a holdout with respect to solver development. It is not a random sample from all magma implications, and its announced competition distribution may resemble the public suites, but unlike those suites no row could have driven a solver change.

The simultaneously released, unscored order-five research set gives the opposite result. It contains 100 research-tier problems with an order-five Austin law on at least one side, partly unknown ground truth, and no finite countermodel for the relevant false directions. The artifact certified 0/100 within budget: 73 searches exhausted the deep budget and 27 candidate certificates were rejected by the kernel. This

negative ledger (SHA-256 `86fc2764fdb9...`) prevents the holdout result from being read as broad completeness.

8 Honest Boundary and Reproducibility

8.1 Scope of the empirical claims

The scientific claims do not depend on a leaderboard score or rank. Public and distribution-drill rows measure regression, the hosted playground measures toolchain compatibility, and Section 7.6 supplies the temporal holdout; none of them is a completeness theorem or a cross-system comparison. The coverage is bounded by construction: structured countermodels cover chosen families, finite-model search has carrier and time bounds, and superposition deepens only until its deadline. Which problems the deep stages reach can vary with machine load; the judge's acceptance of a fixed certificate under a pinned toolchain does not.

8.2 Threats to validity

The public suites influenced solver development. The fourteen v1 residuals motivated particular prover and countermodel changes, and the E168 drill motivated the fixed central groupoid. Full regression shows that those changes integrate without losing earlier rows; it does not measure performance on an independently sampled corpus. The distribution drill is broader but is still published Stage 1 data, and its ground truth may share construction patterns with the public suites.

Wall-clock totals combine Python search, process overhead, and Lean 4 checking under one operator-host environment. They are appropriate for detecting large regressions within that setup but should not be read as portable runtimes. Parallel worker load affects time-budgeted deepening, and filesystem caching affects Lean 4 startup. The hosted-playground mean times partially address this environment gap, but playground selection and daily interaction are not the same protocol as final evaluation.

Finally, kernel checking validates each emitted theorem relative to the judge's imported environment and allowed axioms. It does not validate that the benchmark labels, competition problem generator, or provenance narrative are correct. We mitigate the last concern with hashes and exact commands; benchmark construction and judge implementation remain external assumptions.

9 Related Work

Equational theories. The Equational Theories Project builds a collaborative implication graph for single magma identities and supplies formal proofs, finite models, and curated problem data [4]. The SAIR task derives its laws and implication setting from this project but imposes a distinct online solver and certificate interface. Cazares analyzes Stage 1 of the challenge and its prediction setting [5]. Stage 2 changes the observable output from a label to a judge-accepted proof or countermodel.

Knuth's study of central groupoids underlies the E168 witness family [11]; the use here is narrower than that structural theory, embedding one checked non-natural finite model because natural constructions separated none of the goals encountered. The Austin models likewise draw on constructions represented in the Equational Theories Project rather than proposing a general finite-model theorem.

Proof-producing superposition and reconstruction. Duper is the closest direct precedent, implementing proof-producing superposition as a Lean tactic for dependent type theory [7]; it is broader in logic and tighter in proof-assistant integration, and sets the positive baseline here. The complementary point studied here couples a narrower equality compiler to finite and infinite negative witness compilers under one dependency policy and a single-file scheduler.

Lean-Auto and Lean on Vampire translate Lean goals to external provers and reconstruct their answers [3, 16], answering the natural alternative "call a mature ATP, then replay." That architecture supplies a stronger general prover; the deployment contract here admits a single Python file, and specialization buys proof search the budget to share parsing, scheduling, and certificate dispatch with the countermodel branch.

Isabelle's Sledgehammer architecture delegates search to external provers and uses tactics such as Metis for reconstruction [10, 15]. The SLAM tactic instead targets higher-order logic directly with λ-superposition and can itself serve as a Sledgehammer backend [9]. Our positive replay shares the same skeptical principle, but it emits first-order unit-equality chains rather than translating higher-order proofs. TSTP provides a general interchange vocabulary for ATP problems and derivations [20]; our recipes are smaller and domain-specific, but this choice sacrifices interoperability with mature ATP tooling.

Formalized calculi and model finding. Formalizations of superposition study the soundness and completeness of the calculus itself; recent Isabelle work extends such a framework with sorts [22]. Our Python calculus is not formally verified. Instead, each successful derivation is compiled away and checked extensionally as a Lean equality proof. This reduces the trusted base for each answer but does not establish completeness or correctness of the search algorithm.

Vampire [13], E [19], and Prover9/Mace4 [14] implement mature saturation and finite-model architectures; Paradox develops efficient MACE-style finite model finding [6]. Our bounded finder is less general. Its system contribution is that the discovered table is not accepted as a Python-side Boolean: it is embedded in a negative Lean witness and exhaustively checked under the same contract as a positive proof. Structured and infinite families cover cases where generic small-table enumeration is either wasteful or inapplicable.

Lean integration. Section 2 collects the Lean-specific consequences of equality elimination, finite reflection, dependency inspection, and toolchain startup [8, 21]. Together they favor small, locally typed certificates over replaying a global search trace. A reusable length-independent finite-operation certificate API would remove the remaining proof-assistant-specific printer branch.

10 Conclusion

The SAIR EQT2 Stage 2 contract rewards an answer only when the same artifact also supplies a machine-checked reason. The solver described here meets that contract with one fail-closed cascade for both polarities: structured and irregular countermodels, infinite witnesses, and ordered unit superposition all terminate in judge-checked Lean terms. This dual-polarity integration, the restricted single-file deployment, and the evidence discipline distinguish the system from proof-producing superposition alone. Public regressions show integration coverage; the 200/200 temporal holdout tests the byte-identical artifact after freeze, while the research-tier failure marks a concrete frontier. The broader lesson is that aggressive untrusted search remains auditable when each stage preserves enough structure to emit a small positive proof or an explicit negative model, and when empirical claims are bound to the exact bytes that produced them.

Author contributions (CRediT)

Haobo Ma: conceptualization, methodology, software (solver architecture and the systems foundation), investigation, validation. Wenlin Zhang: conceptualization, methodology, software, investigation, validation, data curation, project administration, writing (original draft). Manuel Israel Cázares: validation (solver freeze verification — hash, file size, and single-file contract compliance; evidence-ledger auditing across the reviewed freezes); methodology (review of experimental design and the claims–evidence boundary); writing (review and editing — full manuscript review and critical review of the presentation and claims).

References

[1] A. K. Austin. 1965. A Note on Models of Identities. *Proc. Amer. Math. Soc.* 16 (1965), 522–523.

[2] Leo Bachmair and Harald Ganzinger. 1994. Rewrite-Based Equational Theorem Proving with Selection and Simplification. *Journal of Logic and Computation* 4, 3 (1994), 217–247.

[3] Jonas Bodingbauer, Márton Hajdu, Laura Kovács, Axel Polaczek, and Michael Rawson. 2026. Lean on Vampire Proofs. In *Interactive Theorem Proving (ITP 2026) (Leibniz International Proceedings in Informatics, Vol. 382)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 36:1–36:9. doi:10.4230/LIPIcs.ITP.2026.36

[4] Matthew Bolan, Joachim Breitner, Jose Brox, Nicholas Carlini, Mario Carneiro, Floris van Doorn, et al. 2025. The Equational Theories Project: Advancing Collaborative Mathematical Research at Scale. arXiv preprint arXiv:2512.07087. arXiv:2512.07087 Project repository: https://github.com/teorth/equational_theories.

[5] Manuel Israel Cázares. 2026. Less Is More: Cognitive Load and the Single-Prompt Ceiling in LLM Mathematical Reasoning. arXiv preprint arXiv:2604.18897. arXiv:2604.18897

[6] Koen Claessen and Niklas Sörensson. 2003. New Techniques that Improve MACE-Style Finite Model Finding. In *Proceedings of the CADE-19 Workshop on Model Computation*.

[7] Joshua Clune, Yicheng Qian, Alexander Bentkamp, and Jeremy Avigad. 2024. Duper: A Proof-Producing Superposition Theorem Prover for Dependent Type Theory. In *Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics, Vol. 309)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 10:1–10:20. doi:10.4230/LIPIcs.ITP.2024.10

[8] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635.

[9] Massin Guerdi. 2026. A Lambda-Superposition Tactic for Isabelle/HOL. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, 117–127. doi:10.1145/3779031.3779093

[10] Joe Hurd. 2003. First-Order Proof Tactics in Higher-Order Logic Theorem Provers. In *Design and Application of Strategies/Tactics in Higher Order Logics*. NASA Technical Reports, 56–68.

[11] Donald E. Knuth. 1970. Notes on Central Groupoids. *Journal of Combinatorial Theory* 8 (1970), 376–390.

[12] Donald E. Knuth and Peter B. Bendix. 1970. Simple Word Problems in Universal Algebras. In *Computational Problems in Abstract Algebra (Proc. Conf., Oxford, 1967)*. Pergamon, 263–297.

[13] Laura Kovács and Andrei Voronkov. 2013. First-Order Theorem Proving and Vampire. In *Computer Aided Verification (CAV 2013) (Lecture Notes in Computer Science, Vol. 8044)*. Springer, 1–35.

[14] William McCune. 2005–2010. Prover9 and Mace4. Automated reasoning systems. https://www.cs.unm.edu/~mccune/prover9/

[15] Lawrence C. Paulson and Jasmin Christian Blanchette. 2010. Three Years of Experience with Sledgehammer, a Practical Link between Automatic and Interactive Theorem Provers. In *Proceedings of the 8th International Workshop on the Implementation of Logics*.

[16] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. 2025. Lean-Auto: An Interface Between Lean 4 and Automated Theorem Provers. In *Computer Aided Verification (CAV 2025) (Lecture Notes in Computer Science)*. Springer, 175–196. doi:10.1007/978-3-031-98682-6_10

[17] SAIR Foundation. 2026. Equational Theories Stage 2: Evaluation Setup and Official Repository. Competition rules and reference implementation. https://github.com/SAIRcompetition/equational-theories-lean-stage2

[18] SAIR Foundation, Damek Davis, and Terence Tao. 2026. Mathematics Distillation Challenge: Equational Theories, Stage 2. Official competition overview. https://competition.sair.foundation/

[19] Stephan Schulz. 2002. E – A Brainiac Theorem Prover. *AI Communications* 15, 2–3 (2002), 111–126.

[20] Geoff Sutcliffe. 2009. The TPTP Problem Library and Associated Infrastructure: The FOF and CNF Parts, v3.5.0. *Journal of Automated Reasoning* 43, 4 (2009), 337–362. doi:10.1007/s10817-009-9143-8

[21] The mathlib Community. 2020. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. ACM, 367–381.

[22] Balazs Toth, Martin Desharnais-Schäfer, and Jasmin Blanchette. 2026. Adding Sorts to an Isabelle Formalization of Superposition. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, 104–116. doi:10.1145/3779031.3779099

A Reproducibility Materials

A.1 Hash-bound evidence

`PROVENANCE.json` binds the frozen solver's SHA-256 and byte count, the official repository and configuration revisions, each result-ledger hash, the Marathon outputs, the compatibility corpus, and the hosted-playground record. The human-readable claims ledger states the maximum set of supported claims and separately lists prohibited inferences. The repository script `scripts/check_freeze.py` recomputes the solver and ledger hashes, checks accepted-row metadata, verifies the one-file submission layout, and fails closed on drift.

This structure prevents stale measurements from being attributed to a different solver. Every table uses the v2.5 identity in Table 3; older provenance entries are outside the paper's quantitative claim surface.

A.2 Replay procedure

Reproduction requires this artifact and the official Stage 2 judge at the pinned revision. The artifact-side integrity check is

```
python3 scripts/check_freeze.py
```

The pinned Solo or Marathon runner then consumes a submission directory containing only the frozen `solver.py`. The anonymous supplement includes the exact pinned revisions, manifests, ledgers, and replay commands. Recomputed output must be stored separately rather than overwriting the archived evidence.

The Lean 4 4.32 compatibility corpus was run in an external directory and is described in the provenance record; it is not presently packaged as a one-command repository test. This is a reproducibility limitation. The official harness, freeze checker, ledger files, and public-runner commands are available locally; hosted playground interaction and any future leaderboard submission require organizer infrastructure.